AGENTIC AI ENTERPRISE PLATFORM

Final Model · Build Planner

Production-Grade Enterprise Deployment

The final model closes the response gap. Live ERP/MES integration, real IoT data ingestion, full governance and audit layer, role-based access control, a mobile-ready interface, and cloud deployment. This is the architecture that CIOs, COOs, and CDOs can put in front of their boards.

BUILDS ON	EFFORT	CLOUD	AUDIENCE
All 3 Betas	3–4 weeks	AWS / Azure / GCP	Enterprise buyers

01 WHAT THE FINAL MODEL DELIVERS

Every layer of the architecture described in the vision document is now **operational, governed, and auditable.** The enterprise can detect, decide, approve, and close exceptions across functions faster than disruption hits the business.

Final model capability matrix

- Live ERP integration: SAP / Oracle API connectors pulling real PO, inventory, production data
- MES integration: OPC-UA / MQTT bridge reading machine OEE, downtime, reject rates in real-time
- IoT ingestion pipeline: Kafka streams → Faust processor → enricher → knowledge graph
- Warehouse + logistics: WMS webhook events, 3PL API polling for OTIF tracking
- Full RBAC: roles for Operator, Supervisor, Manager, Executive, Auditor
- Governance panel: every AI recommendation logged with input, output, model version, timestamp
- Audit trail: immutable log of every approval, override, escalation — queryable by role
- Mobile PWA: same React app, responsive layout, push notifications for critical alerts
- Cloud deployment: containerized on AWS ECS or Azure AKS, Postgres RDS, Redis cache
- Analytics module: exception trends, agent performance, MTTR (Mean Time to Resolve) dashboard

02 FULL SYSTEM ARCHITECTURE

Layer	Components	Integration Method
Physical Layer	CNC machines, conveyors, AGVs, sensors, SCADA	OPC-UA, MQTT, Modbus TCP
Supply Network	ERP (SAP/Oracle), Supplier portals, 3PL APIs	REST APIs, EDI, webhooks
Warehouse + Logistics	WMS, TMS, barcode/RFID scanners	WMS webhook, RFID middleware
IoT + Streaming	Kafka cluster, Faust stream processor, Redis	Kafka topics per data type
Integration Layer	API Gateway, message broker, ETL pipelines	Kafka Connect, custom connectors
Knowledge Graph	Neo4j Aura Enterprise, Cypher query engine	Python neo4j driver, REST API
AI Agent Layer	LangGraph multi-agent, 6+ specialized agents	Internal API, shared state graph
LLM Gateway	OpenAI GPT-4o + Claude fallback, prompt registry	Abstraction layer with retry/cache
Governance Layer	Audit DB (Postgres), RBAC, approval workflow	FastAPI middleware, JWT auth
Frontend	React PWA, role-specific dashboards, mobile	REST + WebSocket
Cloud Infra	AWS ECS / Azure AKS, RDS Postgres, Redis, S3	Terraform IaC, GitHub Actions CI/CD

03 FULL FILE / REPO STRUCTURE

Monorepo structure at final stage:

File / Folder	Extension	Purpose
`agentic-platform/`	`(folder)`	Root monorepo
`■■■ frontend/`	`(folder)`	React PWA (extended from Beta 3)
`■ ■■■ src/pages/`	`(folder)`	All role-specific page views
`■ ■ ■■■ OperatorView.jsx`	`.jsx`	Shop-floor operator dashboard
`■ ■ ■■■ ManagerView.jsx`	`.jsx`	Exception management + approvals
`■ ■ ■■■ ExecutiveView.jsx`	`.jsx`	KPI summary + trend analytics
`■ ■ ■■■ AuditorView.jsx`	`.jsx`	Full audit trail with filters

■ ■■■ src/auth/	(folder)	Authentication + RBAC
■ ■ ■■■ AuthContext.jsx	.jsx	JWT token management
■ ■ ■■■ ProtectedRoute.jsx	.jsx	Role-gated route wrapper
■■■ backend/	(folder)	FastAPI server (extended)
■ ■■■ integrations/	(folder)	External system connectors
■ ■ ■■■ sap_connector.py	.py	SAP RFC / OData API client
■ ■ ■■■ oracle_connector.py	.py	Oracle REST API client
■ ■ ■■■ opcua_client.py	.py	OPC-UA machine data poller
■ ■ ■■■ kafka_producer.py	.py	Publishes ingested events to Kafka
■ ■ ■■■ kafka_consumer.py	.py	Consumes Kafka topics → DB + graph
■ ■■■ governance/	(folder)	Audit + compliance layer
■ ■ ■■■ audit_logger.py	.py	Writes every AI action to audit_log table
■ ■ ■■■ rbac.py	.py	Role definitions + permission checks
■ ■ ■■■ approval_workflow.py	.py	Multi-step approval state machine
■ ■■■ analytics/	(folder)	Reporting + trend engine
■ ■ ■■■ metrics.py	.py	MTTR, exception volume, OEE trend calcs
■ ■ ■■■ reports.py	.py	PDF/Excel report generation endpoints
■■■ infra/	(folder)	Infrastructure as Code
■ ■■■ terraform/	(folder)	AWS/Azure cloud resource definitions
■ ■ ■■■ main.tf	.tf	ECS cluster, RDS, Redis, S3, IAM
■ ■ ■■■ variables.tf	.tf	Environment-specific variables
■ ■ ■■■ outputs.tf	.tf	Output values (URLs, ARNs)
■ ■■■ k8s/	(folder)	Kubernetes manifests (AKS option)
■ ■■■ deployment.yaml	.yaml	App deployment spec
■ ■■■ service.yaml	.yaml	Load balancer service
■ ■■■ ingress.yaml	.yaml	HTTPS ingress config
■■■ .github/workflows/	(folder)	CI/CD pipelines
■ ■■■ test.yml	.yml	Run pytest + frontend lint on PR

■ ■■■ deploy.yml	.yml	Build Docker → push ECR → deploy ECS
■■■ docker-compose.yml	.yml	Local full-stack: API + Neo4j + Kafka + Redis
■■■ README.md	.md	Complete setup + deployment guide

04 GOVERNANCE + HUMAN-IN-LOOP DESIGN

This is the differentiator. Every enterprise buyer will ask: '*How do we maintain control?*' Here is the answer:

01	**Tiered Approval Thresholds** Low-severity: agent auto-resolves and logs. Medium: supervisor approval within 2 hrs. High/Critical: manager approval + escalation timer. >$50K impact: executive approval required.
02	**Immutable Audit Log** Every agent action written to append-only audit_log table with: user_id or agent_id, action_type, input_data_hash, output_data, model_version, timestamp, duration_ms.
03	**Override & Explanation** Any human can override an agent recommendation. Override must include a reason code. Override pattern is fed back into agent prompt context for future similar cases.
04	**Model Version Pinning** Each agent config stores the LLM model version used. Audit trail records this per action. Rollback to previous model version is a one-line config change.
05	**RBAC Enforcement** JWT claims carry role. Every FastAPI endpoint decorated with require_role(). Frontend hides/shows UI elements based on decoded role. Auditor role is read-only across all data.

05 BUILD ROADMAP (4-WEEK SPRINT)

01	**Week 1 — Integration layer** Build SAP/Oracle connector stubs (use mock data if no real access). Set up Kafka locally. OPC-UA poller for machine data. Wire all streams into DB + graph.
02	**Week 2 — Governance + RBAC** Implement JWT auth with role claims. Build audit_logger middleware. Build approval state machine with timer escalation. Wire governance panel to frontend.
03	**Week 3 — Analytics + mobile** Build metrics.py MTTR and trend calculations. Executive dashboard with Recharts. Responsive CSS pass for mobile. PWA manifest + service worker for offline badge.
04	**Week 4 — Cloud deploy + hardening** Write Terraform for chosen cloud provider. Set up GitHub Actions CI/CD. Load test: 500 concurrent exceptions, 10 agents. Fix bottlenecks. Security review: OWASP checklist, secret scanning, dependency audit.

06 ENTERPRISE PITCH READINESS CHECKLIST

Before presenting to CIO / COO / CDO

- Live demo runs end-to-end: exception fires → agent reasons → approval triggered → audit logged
- Architecture diagram matches deployed reality (not aspirational)
- RBAC demo: log in as Operator vs Manager vs Auditor — different views
- Governance demo: show audit trail with agent reasoning, model version, timestamps
- Resilience: kill the LLM API mid-demo → fallback message appears gracefully
- Security: can explain data residency, encryption at rest + in transit, access controls
- ROI framing: MTTR reduction, exception escalation rate, hours saved per week per function
- Roadmap slide: show clear path from current state to full ERP-connected deployment

The question every enterprise buyer will ask

- 'Can your system connect to our SAP?' → Yes, connector layer is designed for this.
- 'Who approved that AI decision?' → Audit trail shows exactly who, when, and why.
- 'What if the AI is wrong?' → Human override is always available, logged, and fed back.
- 'Where does our data go?' → Deployed in your cloud tenant, no data leaves your boundary.
- 'How do we start?' → Pilot on one production line, one exception category, 30 days.